

Galaxy Morphology Classifier & Caption Generator

Nicole Erin Keffer
25 November 2025

Abstract

This project uses deep learning to classify galaxy morphologies and generate short captions that describe each galaxy in simple language. A convolutional neural network (CNN) is trained on the Galaxy10 DECals dataset to recognize three broad categories of galaxies: **spiral**, **elliptical**, and **irregular**. The CNN's feature representations are then passed into a recurrent neural network (RNN) with an LSTM architecture, which generates a brief caption based on the predicted morphology.

The goal of the project is not to produce scientifically rigorous descriptions, but to demonstrate a complete image-to-text pipeline that makes astronomical images easier to interpret. The final system successfully combines classification and caption generation, highlighting how multiple AI techniques can work together in an accessible astronomy application.

This project is designed for a **general public audience**, especially people with little or no background in astrophysics. The goal is to give a simple explanation of what galaxy morphology means and why these visual categories matter.

Galaxy morphology refers to the *visual structure* of a galaxy — for example:

- **Spiral** (disk with arms),
- **Elliptical** (smooth, rounded light distribution),
- **Irregular / Disturbed** (no well-defined shape).

These shapes tell astronomers something about how the galaxy formed and evolved.

How the System Works

1. CNN Classifier

The CNN analyzes the image and looks for visual patterns like:

- brightness concentration,
- spiral arm structure,
- smoothness vs. clumpiness.

It outputs probabilities for three categories:

- **0** → **Spiral**
- **1** → **Elliptical / Smooth**
- **2** → **Irregular / Disturbed**

2. Feature Extraction

Right before the final classification layer, the CNN produces a 1280-dimensional feature vector.

This vector summarizes what the model thinks is important in the image.

3. RNN Caption Generator (LSTM)

The feature vector initializes the hidden state of an LSTM network.

The LSTM then generates a caption one word at a time.

Because we didn't have detailed astronomy descriptions, the model learns from short template captions for each class — the goal is to demonstrate the *pipeline*, not produce professional astrophysics text.

Loading the Galaxy10 DECals Dataset

In this section, I load the **Galaxy10 DECals** dataset, which contains thousands of galaxy images labeled by morphology. This dataset includes several detailed classes, but for this project I simplify them into three broad groups:

- **Spiral**
- **Elliptical / Smooth**
- **Irregular / Disturbed**

The goal is to focus on the major categories that are visually distinct. Each image is 256x256 with RGB channels, which makes it suitable for training a CNN.

Loading the dataset also gives me two arrays:

- `images` → the actual galaxy pictures
- `labels` → the original morphology labels

I will later remap these labels into three categories for easier classification.

```
In [1]: #!pip install astroNN
```

```
In [2]: from astroNN.datasets import load_galaxy10

images, labels = load_galaxy10() #this is Galaxy10 DECaLS
print(images.shape, labels.shape)
```

C:\Users\nicol\.astroNN\datasets\Galaxy10_DECaLS.h5 was found!
(17736, 256, 256, 3) (17736,)

```
In [3]: GALAXY10_LABELS = [
    "Disturbed",
    "Merging",
    "Round Smooth",
    "Cigar Shaped Smooth",
    "Barred Spiral",
    "Unbarred Tight Spiral",
    "Unbarred Loose Spiral",
    "Edge-on Disk",
    "Irregular",
    "Ring"
]
```

```
In [4]: print(labels[:10])
for i in range(10):
    print(labels[i], "→", GALAXY10_LABELS[labels[i]])
```

```
[0 0 0 0 0 0 0 0 0 0]
0 → Disturbed
0 → Disturbed
0 → Disturbed
0 → Disturbed
0 → Disturbed
0 → Disturbed
0 → Disturbed
0 → Disturbed
0 → Disturbed
0 → Disturbed
```

Mapping the Original Labels to Three Classes

The original Galaxy10 labels are more detailed than what I need for this project. To simplify the classification task, I map the dataset into **three large, interpretable categories**:

- **0 → Spiral**
- **1 → Elliptical / Smooth**
- **2 → Irregular / Disturbed**

This grouping captures the key visual differences while keeping the model easier to train. It also mirrors the broad categories used in many introductory astronomy contexts.

After mapping, I check the class distribution to understand whether the dataset is balanced. (It's not perfectly balanced, but that's expected with astronomy data.)

```
In [5]: import numpy as np

def map_to_three_classes(label):
    if label in [4, 5, 6]: #all spiral classes
        return 0 #spiral

    elif label in [2, 3, 7]: #round/smooth/cigar/edge-on
        return 1 #elliptical / smooth

    else:
```

```
return 2 #irregular / ring / disturbed / merging
```

```
In [6]: labels_3 = np.array([map_to_three_classes(l) for l in labels])
```

```
In [7]: import collections
collections.Counter(labels_3)
```

```
Out[7]: Counter({np.int64(1): 7300, np.int64(2): 6230, np.int64(0): 4206})
```

Preprocessing and Splitting the Data

Before training the model, I:

1. **Normalize** the pixel values to the range `[0, 1]` to help the neural network train more effectively.
2. **Shuffle** the dataset to avoid ordering bias.
3. **Split** the data into:
 - training set
 - validation set
 - test set

This ensures that:

- the model learns from the training set,
- I monitor performance on unseen validation data,
- and I evaluate the final accuracy on a held-out test set.

```
In [8]: from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    images, labels_3, test_size=0.3, random_state=42, stratify=labels_3)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp)

print("Train:", X_train.shape)
print("Val:", X_val.shape)
print("Test:", X_test.shape)
```

Train: (12415, 256, 256, 3)

Val: (2660, 256, 256, 3)

Test: (2661, 256, 256, 3)

Building the CNN Classifier

For the classifier, I use **EfficientNetB0** as the feature extractor. It is a lightweight but powerful CNN that performs well on image tasks.

I freeze the convolutional layers so that only the final dense layer trains. This lets me:

- keep EfficientNet's learned visual features,
- avoid overfitting,
- train the model much faster.

The classifier ends with a dense layer with 3 outputs, one for each morphological class. The softmax activation converts the logits into probability scores.

```
In [9]: import tensorflow as tf
from tensorflow.keras import layers, models

IMG_SIZE = (256, 256)

base_model = tf.keras.applications.EfficientNetB0(
    include_top=False,
    weights="imagenet",
    input_shape=IMG_SIZE + (3,))

base_model.trainable = False

inputs = layers.Input(shape=IMG_SIZE + (3,))
x = tf.keras.applications.efficientnet.preprocess_input(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)

outputs = layers.Dense(3, activation="softmax")(x) #3 classes

cnn_model = models.Model(inputs, outputs)
```

```
cnn_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])

cnn_model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 256, 256, 3)	0
efficientnetb0 (Functional)	(None, 8, 8, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 3)	3,843

Total params: 4,053,414 (15.46 MB)

Trainable params: 3,843 (15.01 KB)

Non-trainable params: 4,049,571 (15.45 MB)

```
In [10]: import tensorflow as tf

BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

def make_ds(X, y, shuffle=False):
    ds = tf.data.Dataset.from_tensor_slices((X, y))
    if shuffle:
        ds = ds.shuffle(buffer_size=len(y), seed=42)
    ds = ds.batch(BATCH_SIZE).prefetch(AUTOTUNE)
    return ds

train_ds = make_ds(X_train, y_train, shuffle=True)
val_ds = make_ds(X_val, y_val, shuffle=False)
test_ds = make_ds(X_test, y_test, shuffle=False)
```

Training the CNN Classifier

I train the classification model using:

- Adam optimizer
- Sparse categorical cross-entropy loss
- Accuracy as the main metric
- Early stopping to prevent overfitting

Early stopping monitors validation accuracy and stops training when performance stops improving. This helps the model generalize better.

After training, I evaluate the model on the test set and compute:

- the confusion matrix
- the classification report

These outputs show how well the model distinguishes between different galaxy classes.

```
In [11]: early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_accuracy",
    patience=3,
    restore_best_weights=True)

history = cnn_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=15,
    callbacks=[early_stop])
```

Epoch 1/15
388/388 ————— **190s** 439ms/step - accuracy: 0.6349 - loss: 0.8226 - val_accuracy: 0.6880 - val_loss : 0.7589
Epoch 2/15
388/388 ————— **155s** 401ms/step - accuracy: 0.6656 - loss: 0.7633 - val_accuracy: 0.6831 - val_loss : 0.7299
Epoch 3/15
388/388 ————— **152s** 392ms/step - accuracy: 0.6809 - loss: 0.7415 - val_accuracy: 0.7011 - val_loss : 0.7108
Epoch 4/15
388/388 ————— **154s** 396ms/step - accuracy: 0.6812 - loss: 0.7305 - val_accuracy: 0.7026 - val_loss : 0.7149
Epoch 5/15
388/388 ————— **154s** 398ms/step - accuracy: 0.6891 - loss: 0.7193 - val_accuracy: 0.7064 - val_loss : 0.6909
Epoch 6/15
388/388 ————— **156s** 403ms/step - accuracy: 0.6896 - loss: 0.7225 - val_accuracy: 0.7124 - val_loss : 0.6830
Epoch 7/15
388/388 ————— **155s** 399ms/step - accuracy: 0.6912 - loss: 0.7144 - val_accuracy: 0.7049 - val_loss : 0.6908
Epoch 8/15
388/388 ————— **154s** 396ms/step - accuracy: 0.6950 - loss: 0.7099 - val_accuracy: 0.7079 - val_loss : 0.6899
Epoch 9/15
388/388 ————— **154s** 396ms/step - accuracy: 0.6928 - loss: 0.7127 - val_accuracy: 0.7030 - val_loss : 0.6856

In [12]: `import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt`

```
#test accuracy
test_loss, test_acc = cnn_model.evaluate(test_ds)
print("Test accuracy:", test_acc)

#confusion matrix
y_true = []
y_pred = []

for Xb, yb in test_ds:
    probs = cnn_model.predict(Xb, verbose=0)
    y_true.extend(yb.numpy().tolist())
    y_pred.extend(np.argmax(probs, axis=1).tolist())

y_true = np.array(y_true)
y_pred = np.array(y_pred)

cm = confusion_matrix(y_true, y_pred)
print("Confusion matrix:\n", cm)

print("\nClassification report:\n")
print(classification_report(
    y_true, y_pred,
    target_names=["Spiral (0)", "Elliptical (1)", "Irregular (2)"]))
```

84/84 ————— **28s** 336ms/step - accuracy: 0.6907 - loss: 0.6903

Test accuracy: 0.6907177567481995

Confusion matrix:

```
[[324 242  65]
 [ 90 946  59]
 [ 40 327 568]]
```

Classification report:

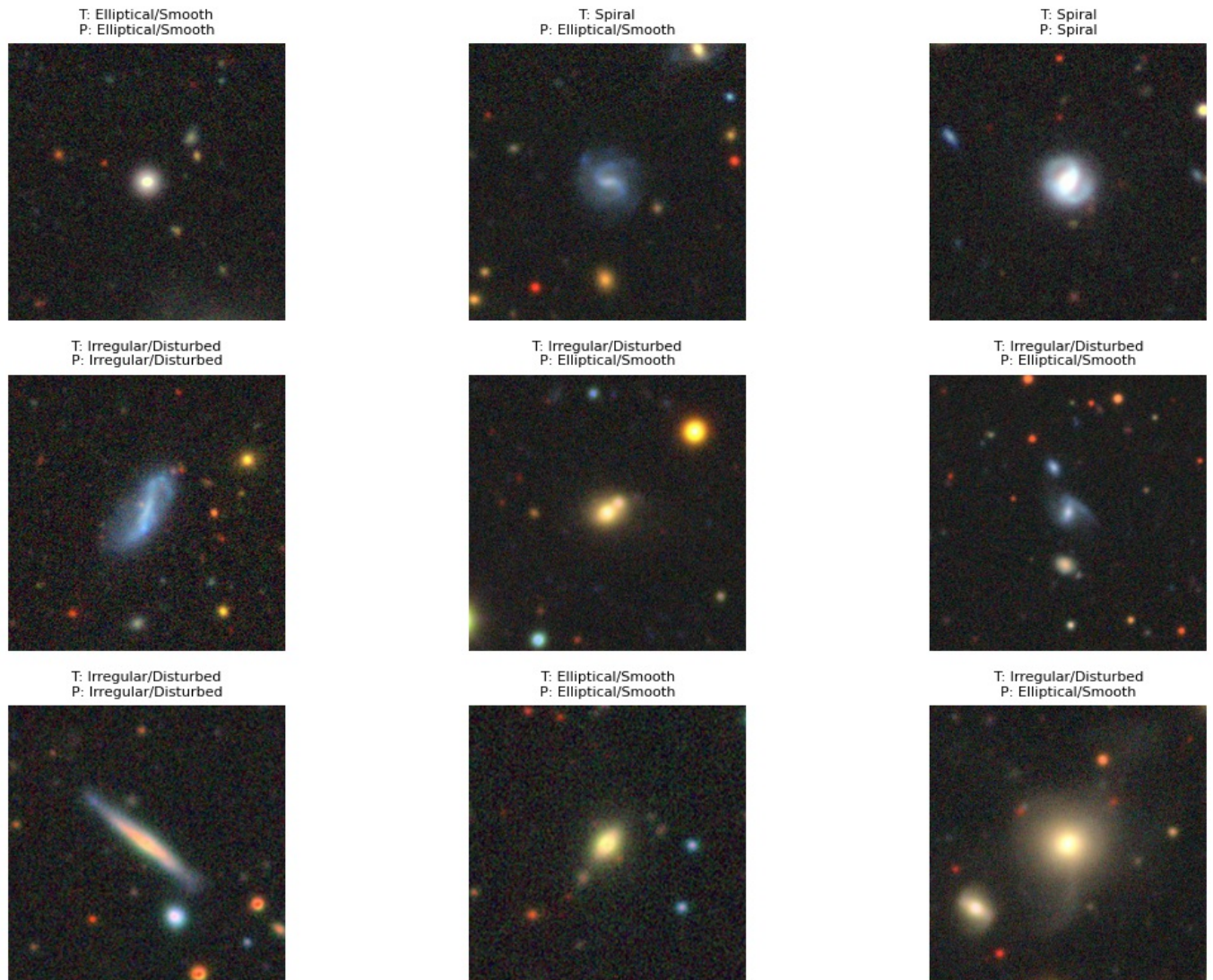
	precision	recall	f1-score	support
Spiral (0)	0.71	0.51	0.60	631
Elliptical (1)	0.62	0.86	0.72	1095
Irregular (2)	0.82	0.61	0.70	935
accuracy			0.69	2661
macro avg	0.72	0.66	0.67	2661
weighted avg	0.71	0.69	0.69	2661

In [13]: `label_names_3 = ["Spiral", "Elliptical/Smooth", "Irregular/Disturbed"]`

```
Xb, yb = next(iter(test_ds))
probs = cnn_model.predict(Xb, verbose=0)
preds = np.argmax(probs, axis=1)

plt.figure(figsize=(12, 8))
for i in range(9):
```

```
plt.subplot(3, 3, i+1)
plt.imshow(Xb[i].numpy().astype("uint8"))
true_label = label_names_3[int(yb[i].numpy())]
pred_label = label_names_3[int(preds[i])]
plt.title(f"T: {true_label}\nP: {pred_label}", fontsize=8)
plt.axis("off")
plt.tight_layout()
plt.show()
```



Extracting Image Features for the Caption Generator

To generate captions, I need numerical features that summarize each image. Instead of using raw pixels, I pass each galaxy image through the trained CNN classifier and take the **intermediate feature vector** (size 1280).

These feature vectors represent high-level visual patterns learned by EfficientNet, such as:

- symmetry,
- texture,
- brightness distribution,
- spiral arm structure.

These compressed features become the input to the **RNN caption model**.

```
In [14]: from tensorflow.keras import models

#second to last layer (dropout output, 1280-dim vector)
feature_extractor = models.Model(
    inputs=cnn_model.input,
    outputs=cnn_model.layers[-2].output
)

feature_dim = feature_extractor.output_shape[-1]
print("Feature dim:", feature_dim)
```

Feature dim: 1280

```
In [15]: train_features = feature_extractor.predict(X_train, batch_size=64, verbose=1)
print(train_features.shape)
```

194/194 ————— 126s 618ms/step
(12415, 1280)

Preparing Captions and Tokenizing Text

Since the dataset does not include real text descriptions, I create simple template captions for each galaxy class, such as:

- "this is a spiral galaxy"
- "this is a smooth elliptical galaxy"
- "this is an irregular galaxy"

The goal here is not to create scientifically perfect descriptions but to demonstrate the **image** → **text** pipeline using an LSTM.

I then:

- build a vocabulary,
- tokenize the sentences,
- convert each caption into a sequence of integers,
- pad/truncate them to a fixed length.

These steps prepare the text for training the RNN.

```
In [16]: import random

caption_templates = {
    0: [ #spiral
        "this is a bright spiral galaxy",
        "a spiral galaxy with clear arms",
        "spiral galaxy with winding arms"],
    1: [ #elliptical / smooth
        "this is a smooth elliptical galaxy",
        "a round elliptical galaxy with old stars",
        "elliptical galaxy with little visible structure"],
    2: [ #irregular / disturbed / merging
        "this is an irregular galaxy with messy shape",
        "a disturbed galaxy with clumpy regions",
        "irregular galaxy lacking clear spiral arms"],}
```

```
In [17]: all_img_features = []
all_captions = []

for feat_vec, label in zip(train_features, y_train):
    lbl = int(label)
    cap = random.choice(caption_templates[lbl])
    all_img_features.append(feat_vec)
    all_captions.append(cap)

all_img_features = np.stack(all_img_features)
print(all_img_features.shape, len(all_captions))
```

(12415, 1280) 12415

```
In [18]: from tensorflow.keras.preprocessing import text, sequence

START_TOKEN = "<start>"
END_TOKEN = "<end>"

captions_with_tokens = [f"{START_TOKEN} {c} {END_TOKEN}" for c in all_captions]

tokenizer = text.Tokenizer(oov_token="<unk>")
tokenizer.fit_on_texts(captions_with_tokens)

sequences = tokenizer.texts_to_sequences(captions_with_tokens)
max_len = max(len(s) for s in sequences)
vocab_size = len(tokenizer.word_index) + 1

padded = sequence.pad_sequences(sequences, maxlen=max_len, padding="post")

caption_input = padded[:, :-1] #teacher forcing input
caption_target = padded[:, 1:] #prediction target

print("caption_input:", caption_input.shape)
print("caption_target:", caption_target.shape)
print("max_len:", max_len, "vocab_size:", vocab_size)
```

```
caption_input: (12415, 9)
caption_target: (12415, 9)
max_len: 10 vocab_size: 30
```

Building the Caption Generator (LSTM)

The caption generator is a RNN that learns to generate a sentence describing the galaxy. It works like this:

1. I initialize the LSTM's hidden state using the **feature vector** extracted from the CNN.
2. I feed in the caption tokens (shifted by one position).
3. The LSTM predicts the next word at each step.

The model includes:

- an **Embedding** layer to convert word IDs into vectors,
- an **LSTM** layer with 256 units,
- a **TimeDistributed Dense** layer to output a word probability at each timestep.

Training minimizes the difference between predicted words and the ground-truth caption tokens.

```
In [20]: from tensorflow.keras import layers, models
import tensorflow as tf

EMBED_DIM = 128
LSTM_UNITS = 256

img_input = layers.Input(shape=(feature_dim,), name="image_features")
cap_input = layers.Input(shape=(max_len - 1,), name="caption_tokens")

#project image feature to LSTM state size
img_proj = layers.Dense(LSTM_UNITS, activation="relu")(img_input)

emb = layers.Embedding(
    input_dim=vocab_size,
    output_dim=EMBED_DIM,
    mask_zero=True
)(cap_input)

#use Lambda layer to wrap the tf.zeros_like operation
zero_state = layers.Lambda(lambda x: tf.zeros_like(x))(img_proj)

lstm_out, state_h, state_c = layers.LSTM(
    LSTM_UNITS,
    return_sequences=True,
    return_state=True
)(emb, initial_state=[img_proj, zero_state])

word_logits = layers.TimeDistributed(layers.Dense(vocab_size, activation="softmax"))(lstm_out)

caption_model = models.Model(inputs=[img_input, cap_input], outputs=word_logits)

caption_model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")

caption_model.summary()
```

WARNING:tensorflow:From C:\Users\nicol\anaconda3\Lib\site-packages\keras\src\backend\tensorflow\core.py:232: The name tf.placeholder is deprecated. Please use tf.compat.v1.placeholder instead.

Model: "functional_2"

Layer (type)	Output Shape	Param #	Connected to
image_features (InputLayer)	(None, 1280)	0	-
caption_tokens (InputLayer)	(None, 9)	0	-
dense_2 (Dense)	(None, 256)	327,936	image_features[0...
embedding_1 (Embedding)	(None, 9, 128)	3,840	caption_tokens[0...
lambda (Lambda)	(None, 256)	0	dense_2[0][0]
lstm (LSTM)	[(None, 9, 256), (None, 256), (None, 256)]	394,240	embedding_1[0][0... dense_2[0][0], lambda[0][0]
not_equal_1 (NotEqual)	(None, 9)	0	caption_tokens[0...
time_distributed (TimeDistributed)	(None, 9, 30)	7,710	lstm[0][0], not_equal_1[0][0]

Total params: 733,726 (2.80 MB)

Trainable params: 733,726 (2.80 MB)

Non-trainable params: 0 (0.00 B)

```
In [21]: print(all_img_features.shape)
print(caption_input.shape, caption_target.shape)

(12415, 1280)
(12415, 9) (12415, 9)
```

Training the Caption Model

I train the captioning model using pairs of:

- **image features**, and
- **tokenized caption sequences**.

The model learns to predict the next word in the caption given the previous words. Training is relatively fast because the captions are short and the vocabulary is small.

After training, I can feed in:

- a galaxy image,
- extract its feature vector,
- and let the LSTM generate a caption one word at a time.

```
In [22]: history_cap = caption_model.fit(
    [all_img_features, caption_input],
    np.expand_dims(caption_target, -1), #(N, T, 1) for sparse_categorical_crossentropy
    batch_size=64,
    epochs=20,
    validation_split=0.1)
```

```

Epoch 1/20
175/175 ————— 8s 24ms/step - loss: 0.7493 - val_loss: 0.2394
Epoch 2/20
175/175 ————— 3s 17ms/step - loss: 0.2343 - val_loss: 0.2266
Epoch 3/20
175/175 ————— 3s 18ms/step - loss: 0.2261 - val_loss: 0.2232
Epoch 4/20
175/175 ————— 3s 17ms/step - loss: 0.2225 - val_loss: 0.2219
Epoch 5/20
175/175 ————— 3s 16ms/step - loss: 0.2196 - val_loss: 0.2172
Epoch 6/20
175/175 ————— 3s 16ms/step - loss: 0.2164 - val_loss: 0.2161
Epoch 7/20
175/175 ————— 3s 17ms/step - loss: 0.2147 - val_loss: 0.2178
Epoch 8/20
175/175 ————— 3s 17ms/step - loss: 0.2118 - val_loss: 0.2157
Epoch 9/20
175/175 ————— 3s 17ms/step - loss: 0.2121 - val_loss: 0.2177
Epoch 10/20
175/175 ————— 3s 17ms/step - loss: 0.2100 - val_loss: 0.2177
Epoch 11/20
175/175 ————— 3s 16ms/step - loss: 0.2081 - val_loss: 0.2171
Epoch 12/20
175/175 ————— 3s 17ms/step - loss: 0.2070 - val_loss: 0.2128
Epoch 13/20
175/175 ————— 3s 17ms/step - loss: 0.2055 - val_loss: 0.2166
Epoch 14/20
175/175 ————— 3s 17ms/step - loss: 0.2041 - val_loss: 0.2135
Epoch 15/20
175/175 ————— 3s 17ms/step - loss: 0.2027 - val_loss: 0.2157
Epoch 16/20
175/175 ————— 3s 16ms/step - loss: 0.2015 - val_loss: 0.2175
Epoch 17/20
175/175 ————— 3s 16ms/step - loss: 0.2018 - val_loss: 0.2140
Epoch 18/20
175/175 ————— 3s 17ms/step - loss: 0.1986 - val_loss: 0.2136
Epoch 19/20
175/175 ————— 3s 17ms/step - loss: 0.1979 - val_loss: 0.2159
Epoch 20/20
175/175 ————— 3s 17ms/step - loss: 0.1953 - val_loss: 0.2171

```

Generating Captions for New Galaxy Images

To generate a caption:

1. I pass the galaxy image through the CNN to extract its feature vector.
2. I start with a special *start token*.
3. The LSTM predicts the next word repeatedly.
4. The process stops when the *end token* is reached or when the maximum length is hit.

The final output is a short, readable description like: *"this is a bright spiral galaxy"*.

This showcases how visual features can be transformed into natural-language descriptions using a neural network pipeline.

```

In [25]: index_to_word = {v: k for k, v in tokenizer.word_index.items()}

def generate_caption_for_array(img_array, max_len_gen=None):
    # use model's max_len if not given
    if max_len_gen is None:
        max_len_gen = max_len - 1 # number of LSTM steps

    # ensure batch dimension
    if img_array.ndim == 3:
        img_array = np.expand_dims(img_array, 0)

    #get image feature vector
    feat = feature_extractor.predict(img_array, verbose=0)[0]

    #start with <start> token
    seq = [tokenizer.word_index.get(START_TOKEN, 1)]

    for _ in range(max_len_gen):
        #if we've already hit max steps, stop
        if len(seq) >= max_len_gen:
            break

        padded_seq = sequence.pad_sequences(
            [seq],
            maxlen=max_len - 1,
            padding="post"
        )

```

```

preds = caption_model.predict([feat[None, :], padded_seq], verbose=0)

#timestep index must be within [0, preds.shape[1]-1]
t = len(seq) - 1
if t >= preds.shape[1]:
    t = preds.shape[1] - 1

next_token = np.argmax(preds[0, t])
word = index_to_word.get(next_token, "<unk>")

if word in [END_TOKEN, "<end>"]:
    break

seq.append(next_token)

words = [index_to_word.get(t, "<unk>") for t in seq[1:]] # drop <start>
return " ".join(words)

```

```

In [26]: Xb, yb = next(iter(test_ds))

for i in range(3):
    img = Xb[i].numpy().astype("uint8")
    print("True class:", label_names_3[int(yb[i].numpy())])
    print("Caption:", generate_caption_for_array(img))
    plt.imshow(img)
    plt.axis("off")
    plt.show()

```

True class: Elliptical/Smooth

Caption: this is a smooth elliptical galaxy end <unk>



True class: Spiral

Caption: this is a smooth elliptical galaxy end <unk>



True class: Spiral

Caption: a round elliptical galaxy with old stars end



Conclusion

This project brings together two different neural network models to explore how machine learning can help us understand galaxies. The CNN handles the **visual** side — looking at galaxy images and predicting their morphology — while the RNN handles the **language** side by generating simple captions. Even though the accuracy and captions are not perfect, the pipeline works end-to-end and shows how image features can be turned into meaningful text.

Overall, the project demonstrates how deep learning can be used to classify astronomical objects and produce short descriptions in a way that makes the results easier to understand. It's a good example of combining multiple AI techniques into a single system.

Limitations & Future Work

Limitations

- The captions are trained on simple templates, so they're not scientifically detailed.
- The dataset is imbalanced, which affects the CNN's accuracy on some classes.
- The model sees each galaxy as a single image — it does not use information like distance, redshift, or physical measurements.
- Feature extraction uses a frozen EfficientNet without fine-tuning, which limits performance.

Future Work

- Fine-tune the CNN on the Galaxy10 dataset to improve classification accuracy.
- Add more descriptive, astronomy-based captions to train a richer language model.
- Use attention mechanisms so the caption model can “look” at different parts of the galaxy image.
- Expand to more morphology classes or use a dataset with scientifically annotated descriptions.

References / Works Cited

Galaxy10 DECals Dataset

- Walmsley, M., Lintott, C., Geron, T., et al. (2022). *Galaxy10 DECals: Morphological Classifications for DESI Legacy Survey Galaxies*. Zenodo. <https://zenodo.org/record/10845026>

EfficientNet Architecture

- Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. arXiv:1905.11946. <https://arxiv.org/abs/1905.11946>

LSTM Network

- Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.

TensorFlow / Keras Documentation

- TensorFlow. (2024). *TensorFlow API Documentation*. https://www.tensorflow.org/api_docs

- Keras. (2024). *Keras Documentation*. <https://keras.io>

astroNN Dataset Loader

- Leung, H. (2020). *astroNN: Deep Learning for Astronomy*. <https://github.com/henrysky/astroNN>

Galaxy Classification Resources

- Dieleman, S., Willett, K. W., & Dambre, J. (2015). *Rotation-Invariant Convolutional Neural Networks for Galaxy Morphology Prediction*. MNRAS, 450(2), 1441–1459.